

Mid-Semester Report

PINNs and Their Applications

This report summarizes the mid-semester progress of a project on **physics-informed neural networks (PINNs)** for fractional differential and fractional integro-differential equations. The focus is on the mathematical theory, numerical formulation, solver development across multiple branches, and the main results obtained so far.

Student Name: Abdullah Haatim
Project Title: PINNs and Their Applications
Supervisors: Dr Sainath Bitragunta, Dr Sudarshan Santra
Repository: <https://github.com/haatim10/PINNs>
Report Type: Mid-semester progress report

1 Abstract

This project studies **physics-informed neural networks (PINNs)** for solving differential equations with nonlocal and singular structure. The main equation considered in the current phase is a **time-fractional integro-differential equation** with a Caputo fractional derivative, a variable-coefficient diffusion term, and a weakly singular Volterra memory integral. Such problems are important because they model systems with memory, anomalous transport, and hereditary behavior, but they are numerically difficult due to full-history dependence and singular kernels.

The work completed so far develops a solver that combines a graded temporal mesh, an L1 discretization of the Caputo derivative, automatic differentiation for spatial derivatives, and a neural-network surrogate for the unknown solution field. A major development in this semester has been the transition from a simpler baseline quadrature treatment of the Volterra term to a **kernel-aware product integration** strategy that handles the singular kernel more faithfully. Alongside that, earlier branches using a Mexican-hat-activation PINN were also studied and provide valuable benchmark evidence for the representational ability of the network architecture.

Overall, the mid-semester outcome is that the project now has a strong theory-driven PINN framework, successful numerical runs across multiple stages, and a documented path from baseline models to more mathematically faithful formulations. This creates a solid foundation for the next phase of work on future architectures and applications.

2 Introduction and Motivation

Physics-informed neural networks are neural networks trained not only on data but also on the governing laws of a physical system. Instead of learning from labeled examples alone, a PINN is trained so that its output approximately satisfies a differential equation, its boundary conditions, and its initial conditions. This makes PINNs attractive when data are limited but the mathematical model is known.

The present project is especially concerned with problems that are difficult even for many classical numerical methods. In a fractional model, the state at time t depends on its entire previous history rather than only the local derivative at that instant. In a fractional integro-differential model, the equation includes an additional memory integral, and in this project that integral is weakly singular. Therefore the solver must simultaneously handle nonlocal time dependence, singular behavior near the initial time, and a computationally expensive residual evaluation.

The larger goal of the project is broader than one single PDE. It is to study *PINNs and their applications*, with the current semester focusing on fractional models as a mathematically rich testbed. The code branches developed so far show how a theory-guided PINN can evolve from a simpler proof-of-concept solver to a more robust numerical framework.

3 Problem Formulation

3.1 Governing equation

The main model problem studied in the current branch is

$$D_t^\alpha u(x, t) - (x^2 + 1)u_{xx}(x, t) + \int_0^t \sin(x)(t - s)^{-\beta} u(x, s) ds = f(x, t), \quad (1)$$

for $x \in [0, 1]$, $t \in (0, 1]$, and $0 < \alpha, \beta < 1$.

3.2 Boundary and initial conditions

The problem uses non-homogeneous boundary conditions

$$u(0, t) = t^\alpha, \quad u(1, t) = -t^\alpha, \quad (2)$$

and initial condition

$$u(x, 0) = 0. \quad (3)$$

3.3 Manufactured exact solution

To measure accuracy clearly, the implementation uses the exact solution

$$u(x, t) = t^\alpha \cos(\pi x). \quad (4)$$

This solution is useful because it is simple enough for analytic differentiation but still nontrivial in both variables. It also has reduced regularity near $t = 0$, which makes it a meaningful benchmark for fractional solvers.

A useful observation from the README is that the solution shape is naturally symmetric about $x = 0.5$. This comes from the fact that $\cos(\pi x)$ over $[0, 1]$ spans half a cosine period: it is positive at $x = 0$, zero at $x = 0.5$, and negative at $x = 1$. The resulting “mirror-like” field shape seen in the plots is therefore a real property of the manufactured solution rather than a numerical artifact.

3.4 Source term derivation

Using the exact solution above, each term can be computed analytically:

$$D_t^\alpha [t^\alpha \cos(\pi x)] = \Gamma(\alpha + 1) \cos(\pi x), \quad (5)$$

$$(x^2 + 1)u_{xx}(x, t) = -(x^2 + 1)\pi^2 t^\alpha \cos(\pi x), \quad (6)$$

$$\int_0^t \sin(x)(t-s)^{-\beta} s^\alpha \cos(\pi x) ds = \sin(x) \cos(\pi x) t^{\alpha+1-\beta} \frac{\Gamma(\alpha+1)\Gamma(1-\beta)}{\Gamma(\alpha+2-\beta)}. \quad (7)$$

Hence

$$f(x, t) = \cos(\pi x) \left[\Gamma(\alpha + 1) + (x^2 + 1)\pi^2 t^\alpha + \sin(x) t^{\alpha+1-\beta} \frac{\Gamma(\alpha+1)\Gamma(1-\beta)}{\Gamma(\alpha+2-\beta)} \right]. \quad (8)$$

4 Mathematical Background

4.1 Caputo fractional derivative

For $0 < \alpha < 1$, the Caputo fractional derivative is defined by

$$D_t^\alpha u(x, t) = \frac{1}{\Gamma(1-\alpha)} \int_0^t \frac{\partial u(x, s)}{\partial s} (t-s)^{-\alpha} ds. \quad (9)$$

Its key feature is **memory**: evaluating the derivative at time t_n requires information from all earlier times. This is one of the main reasons fractional PDEs are much more expensive than local PDEs. A second important property is that the Caputo derivative behaves naturally with classical initial conditions, which makes it convenient in applications.

4.2 Weakly singular Volterra integral

The Volterra term

$$I(x, t) = \int_0^t \sin(x)(t-s)^{-\beta} u(x, s) ds \quad (10)$$

contains the kernel $(t-s)^{-\beta}$. Since $0 < \beta < 1$, the kernel is singular but still integrable. This means the integral exists, but straightforward quadrature can lose accuracy near $s = t$ because the integrand changes rapidly there.

4.3 Why this benchmark is still difficult

Even though the exact solution is known, the numerical problem remains challenging for three reasons. First, the factor t^α creates a steep gradient near the initial time. Second, the Caputo derivative is nonlocal and requires full time history. Third, the Volterra kernel is singular near the upper integration limit. A reliable PINN for this equation therefore needs more than a standard residual-based formulation.

5 Why PINNs Need Additional Numerical Structure Here

A generic PINN is not sufficient for this problem because the PDE residual itself contains nonlocal operators. In this project, the solver must combine several ingredients:

- a neural network for the unknown field $u(x, t)$,
- automatic differentiation for u_x and u_{xx} ,
- a discrete approximation of the Caputo derivative,
- a robust approximation of the weakly singular Volterra integral,
- and carefully chosen collocation points in space and time.

In other words, the neural network is only one part of the method. The rest of the solver is built from numerical-analysis ideas specifically tailored to the structure of the equation.

6 Graded Temporal Mesh

The README emphasizes that the exact solution behaves like t^α near $t = 0$, so its slope becomes very steep for $0 < \alpha < 1$. A uniform time grid would therefore spend too few points near the region where the solution is hardest to resolve. To address this, the project uses a graded temporal mesh:

$$t_n = T \left(\frac{n}{N} \right)^{\beta_{\text{mesh}}}, \quad n = 0, 1, \dots, N, \quad (11)$$

with $\beta_{\text{mesh}} > 1$. In the current branch, $\beta_{\text{mesh}} = 2.0$.

This grading concentrates many more time nodes near the initial time. As a result, the L1 discretization and the integral approximation both have a better chance of capturing the singular early-time behavior. This is a good example of how the project blends PINNs with classical numerical insight.

7 L1 Discretization of the Caputo Derivative

7.1 L1 formula

The project uses the **L1 scheme** to approximate the Caputo derivative on the graded mesh. At time level t_n ,

$$D_t^\alpha u(x, t_n) \approx d_{n,1}u^n - d_{n,n}u^0 - \sum_{k=1}^{n-1} (d_{n,k} - d_{n,k+1})u^{n-k}, \quad (12)$$

with coefficients

$$d_{n,k} = \frac{(t_n - t_{n-k})^{1-\alpha} - (t_n - t_{n-k+1})^{1-\alpha}}{\Gamma(2-\alpha) \tau_{n-k+1}}. \quad (13)$$

This formula makes the full-history structure explicit: to evaluate the derivative at time level n , the method needs the current value, the initial value, and every intermediate history value.

7.2 Why the L1 scheme is a good fit

The README notes that the L1 scheme is attractive because it handles the singularity at $t = 0$, naturally incorporates memory, and is well suited to fractional PDEs on graded meshes. For a collocation point at time level n , the network must be evaluated at multiple historical times $t_0, t_1, \dots, t_n$. This makes the residual more expensive, but it also means the PINN is explicitly informed by the fractional memory mechanism rather than learning it only indirectly.

Listing 1: Core idea of the L1 history evaluation used in the implementation.

```
for n in unique_n:
    coeffs = self.l1_coeffs.get_coefficients_for_n(n)
    frac_deriv = coeffs[1] * u_n
    for k in range(1, n):
        idx = n - k
        u_idx = model(x, t_nodes[idx])
        diff_coeff = coeffs[k] - coeffs[k + 1]
        frac_deriv -= diff_coeff * u_idx
```

8 Collocation Point Selection and Sampling Strategy

The implementation constructs a tensor-product grid in space and time and then samples interior collocation points from it. The spatial nodes lie in $[0, 1]$, while the temporal nodes come from the graded mesh. The interior grid excludes the spatial boundary and excludes $t = 0$.

A key practical detail from the README is that each collocation point stores not only (x, t) but also the *time index* n . This index is essential because it determines which history terms are needed when assembling the L1 derivative and the memory integral at that point. Boundary-condition samples are taken at $x = 0$ and $x = 1$, and initial-condition samples are taken at $t = 0$.

9 Integral Term Approximation Across Project Stages

9.1 Baseline method before product integration

Before moving to the current branch, the project used a simpler quadrature-style approximation for the weakly singular Volterra term:

$$I(x, t_n) = \int_0^{t_n} \sin(x)(t_n - s)^{-\beta} u(x, s) ds. \quad (14)$$

On each subinterval $[t_j, t_{j+1}]$, the earlier approach used a midpoint-style rule,

$$\int_{t_j}^{t_{j+1}} (t_n - s)^{-\beta} u(x, s) ds \approx (t_n - t_{\text{mid}})^{-\beta} u(x, t_{\text{mid}}) h_j, \quad (15)$$

where $t_{\text{mid}} = (t_j + t_{j+1})/2$ and $h_j = t_{j+1} - t_j$.

This was an important development stage because it established the full PINN workflow and gave the first successful benchmark results. However, the singular kernel itself was still being sampled numerically, which becomes less satisfactory on more demanding runs.

9.2 Kernel-aware product integration

The current branch improves the integral approximation using **product integration**. On each interval $[t_j, t_{j+1}]$, the singular kernel is integrated analytically while the solution factor is approximated linearly:

$$\int_{t_j}^{t_{j+1}} (t_n - s)^{-\beta} u(x, s) ds \approx u(x, t_j) w_j^L + u(x, t_{j+1}) w_j^R. \quad (16)$$

If

$$a = t_n - t_j, \quad b = t_n - t_{j+1}, \quad h_j = t_{j+1} - t_j, \quad (17)$$

then the weights are

$$w_j^L = \frac{1}{h_j} \left[\frac{a^{2-\beta} - b^{2-\beta}}{2-\beta} - b \frac{a^{1-\beta} - b^{1-\beta}}{1-\beta} \right], \quad (18)$$

$$w_j^R = \frac{1}{h_j} \left[a \frac{a^{1-\beta} - b^{1-\beta}}{1-\beta} - \frac{a^{2-\beta} - b^{2-\beta}}{2-\beta} \right]. \quad (19)$$

Thus

$$I(x, t_n) \approx \sin(x) \sum_{j=0}^{n-1} (w_j^L u(x, t_j) + w_j^R u(x, t_{j+1})). \quad (20)$$

The main gain is that the singularity is handled analytically at the interval level, so the numerical approximation focuses on the smoother solution factor. This is the central theoretical upgrade of the current semester.

Listing 2: Representative configuration for the product-integration branch.

```
Nx = 200
Nt = 200
n_collocation = 100
num_epochs = 20000
alpha = 0.5
beta = 0.5
beta_mesh = 2.0
integral_method = "product_integration"
```

10 Neural Network Architecture and Loss Design

The uploaded code and README show that the project uses a fully connected neural network with four hidden layers of width 64 and a Mexican-hat activation:

$$\psi(z) = (1 - z^2)e^{-z^2/2}. \quad (21)$$

The Mexican-hat activation is interesting because it is localized and oscillatory, which can be beneficial when approximating structured solution fields.

The total loss has the standard PINN structure but is adapted to the fractional problem:

$$\mathcal{L} = w_{\text{PDE}}\mathcal{L}_{\text{PDE}} + w_{\text{BC}}\mathcal{L}_{\text{BC}} + w_{\text{IC}}\mathcal{L}_{\text{IC}}. \quad (22)$$

Here, the PDE loss enforces the fractional integro-differential equation, the BC loss enforces non-homogeneous boundary data, and the IC loss enforces the initial condition. In the current branch, the PDE loss is the most complicated term because each residual evaluation includes spatial derivatives, a fractional-history sum, and a history integral.

11 Training Methodology

According to the README, the current product-integration branch uses the following configuration: $N_x = 200$, $N_t = 200$, 100 collocation points per epoch, 20,000 epochs, a peak learning rate of 5×10^{-4} , and warmup over the first 1000 epochs. The integral method is product integration, and the key physical parameters are $\alpha = 0.5$, $\beta = 0.5$, and $\beta_{\text{mesh}} = 2.0$.

The README also highlights a practical outcome of the run: output directories record checkpoints, collocation points, L1-discretization points, and visualization plots. This is useful for a research workflow because it makes the numerical experiments reproducible and makes checkpoint analysis possible.

12 Results by Project Stage

12.1 Main-branch Mexican-hat PINN results

The Mexican-hat branch is an important stage of the project because it demonstrates that the selected network architecture can represent the solution field accurately when coupled with the rest of the PINN machinery. The heatmap, slice plots, and training curves show a stable and accurate learning process.

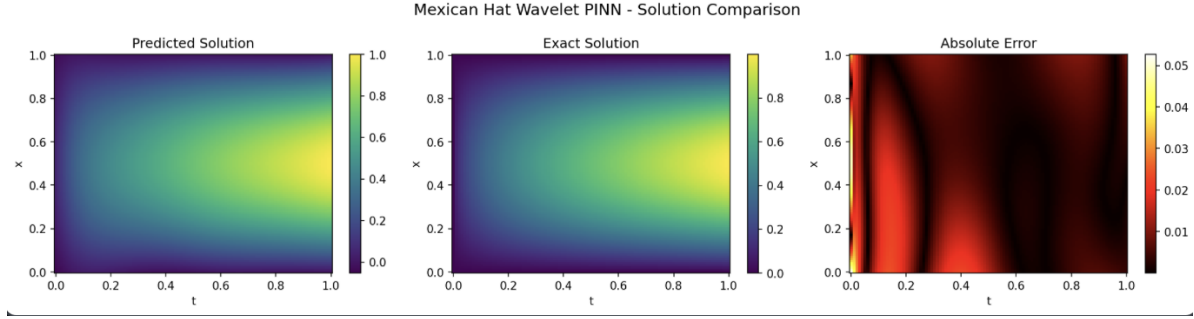


Figure 1: Main-branch Mexican-hat result: predicted solution, exact solution, and absolute error heatmap. The overall field structure is captured well, and the error remains concentrated near the more difficult early-time region.

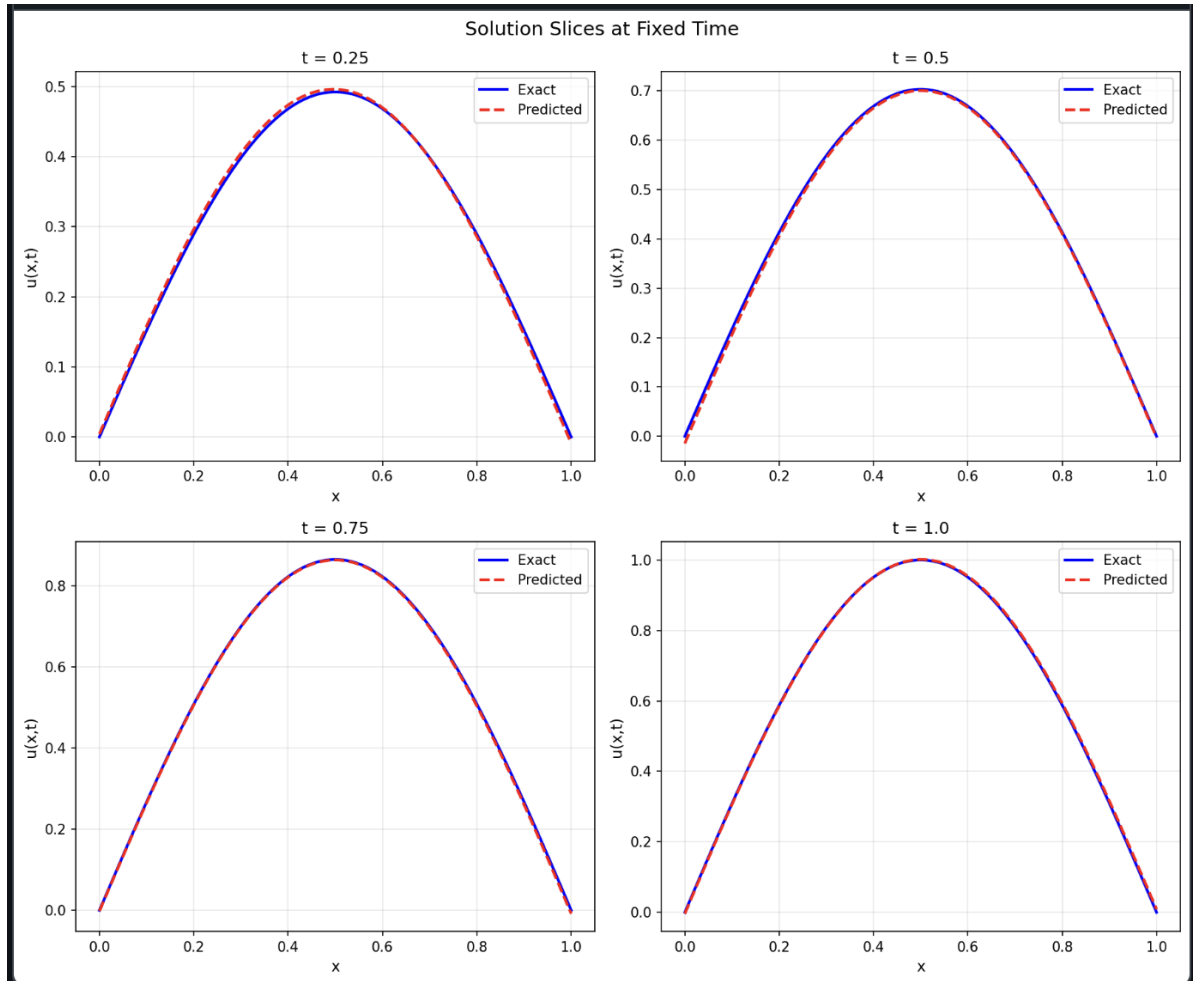


Figure 2: Mexican-hat fixed-time slices. The predicted and exact solutions agree closely across multiple time snapshots, showing that the network captures the spatial structure reliably.

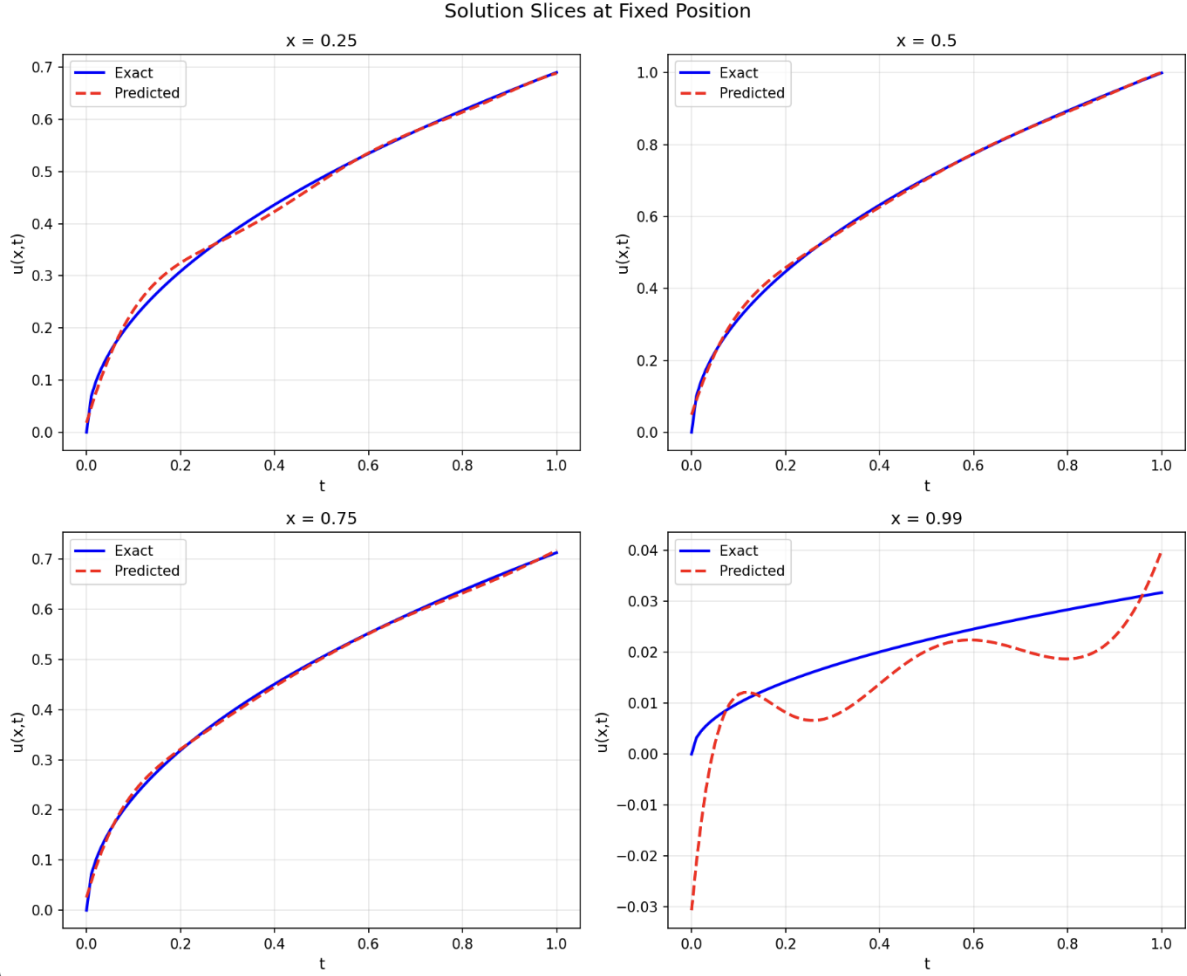


Figure 3: Mexican-hat fixed-position slices. The branch performs well across several spatial locations, while the near-boundary case remains more challenging and therefore informative.

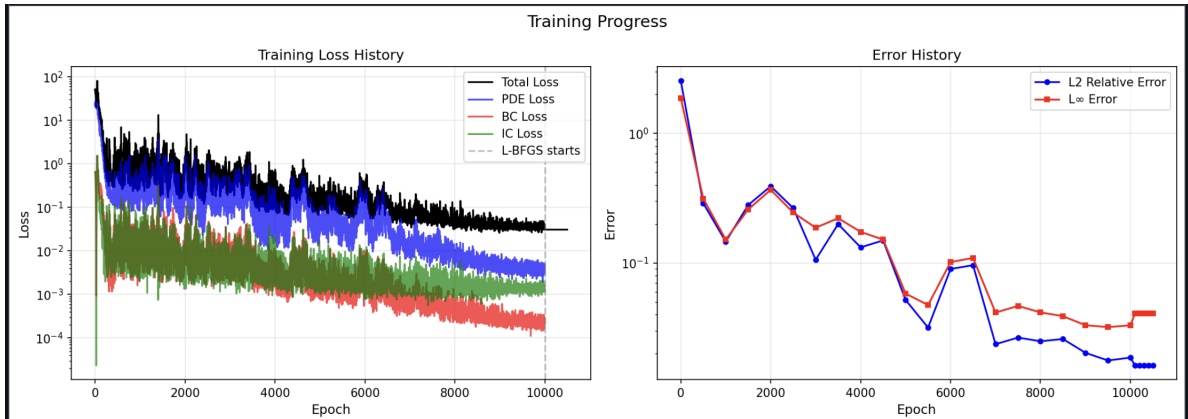


Figure 4: Training progress for the Mexican-hat branch. The plots show the decay of training loss together with the L_2 relative error and L_∞ error across epochs. The overall trend indicates steady convergence and good final accuracy.

12.2 Baseline integro-differential stage without product integration

This earlier solver stage is valuable because it established the integro-differential PINN workflow before the product-integration upgrade. It serves as a baseline benchmark against which the later branch can be interpreted.

The README comparison table reports the following baseline values: a 50×50 grid, simpler midpoint-style integral treatment, about 10,000 epochs, relative L_2 error of 0.54%, L_∞ error of 0.0484, and total training time of roughly 2 hours. These numbers are very good for an earlier-stage run and show that the main PINN pipeline was already functioning well.

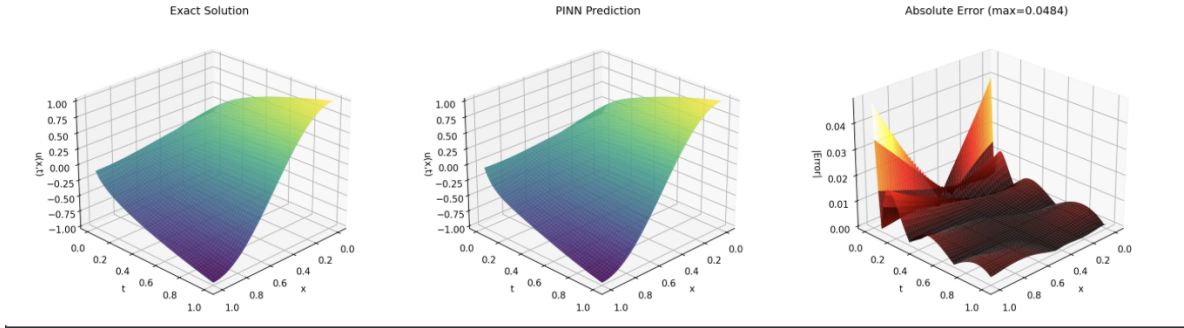


Figure 5: Baseline pre-product-integration three-dimensional plots for the exact solution, PINN prediction, and absolute error. The error remains small overall, but the figure also helps visualize where the earlier approximation begins to struggle.

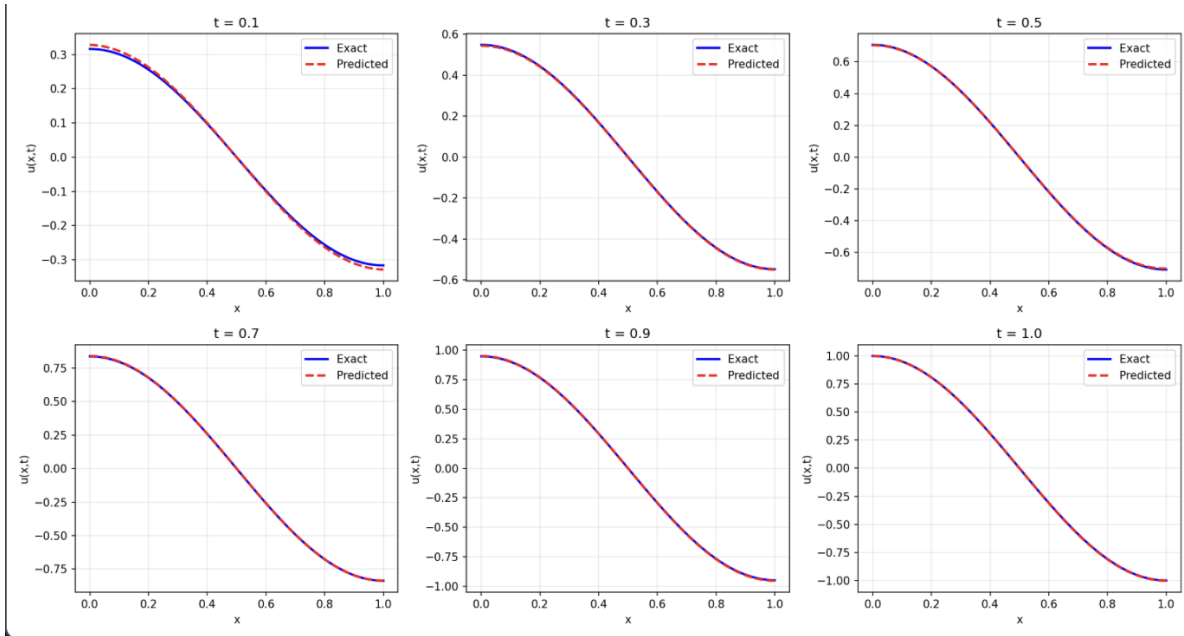


Figure 6: Baseline fixed-time slices before product integration. These curves show that the earlier implementation already captured the main spatial behavior accurately over several time levels.

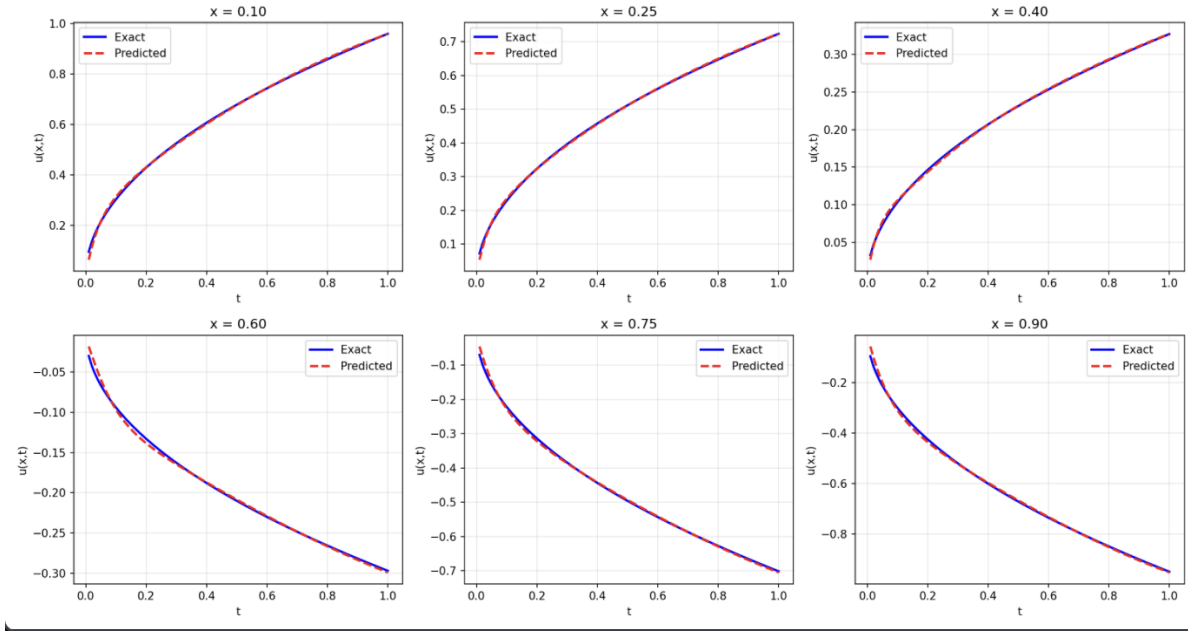


Figure 7: Baseline fixed-position slices before product integration. The mismatch becomes more visible near the difficult edge case around $x = 0.99$, which is useful motivation for the improved integral treatment introduced later.

12.3 Integro-differential PINN with product integration

This stage corresponds to the current enhanced solver. According to the README, the branch solves a **16 times larger problem** than the baseline in the sense of using a 200×200 grid instead of a 50×50 grid. The reported final metrics are a relative L_2 error of **1.52%**, L_∞ error of **0.0606**, mean error of 0.0050, and training time of approximately 32 hours.

The README emphasizes that the larger error should not be interpreted as a regression in a simplistic sense. The current run is much harder: the grid is much finer, the history evaluation is denser, and the Volterra term is treated with a more faithful product-integration method. The real advance is the improved mathematical treatment of the weakly singular kernel.

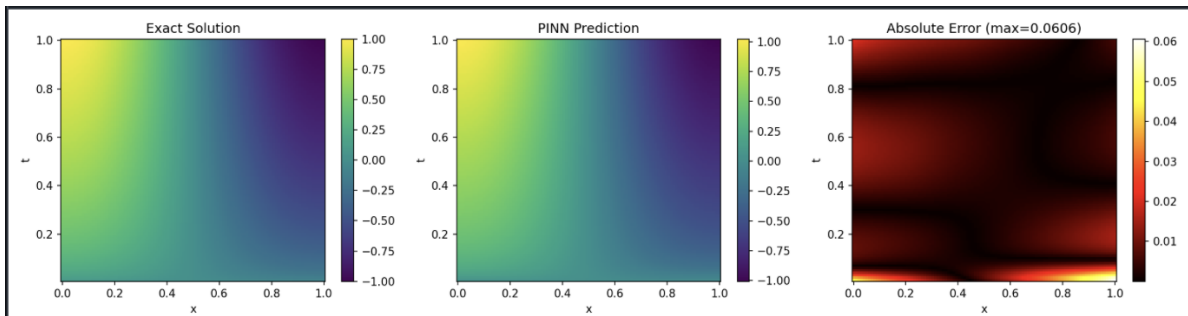


Figure 8: Product-integration result: exact solution, PINN prediction, and absolute error heatmap for the integro-differential problem. The reported maximum pointwise error is approximately 0.0606, and the largest discrepancies remain concentrated near the most difficult regions.

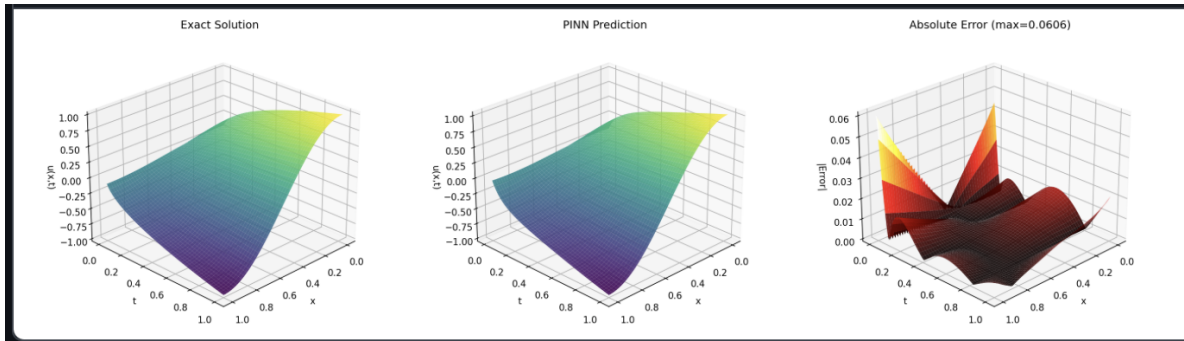


Figure 9: Three-dimensional visualization for the product-integration run. The exact and learned surfaces have nearly identical global shape, while the error surface indicates where residual minimization is hardest.

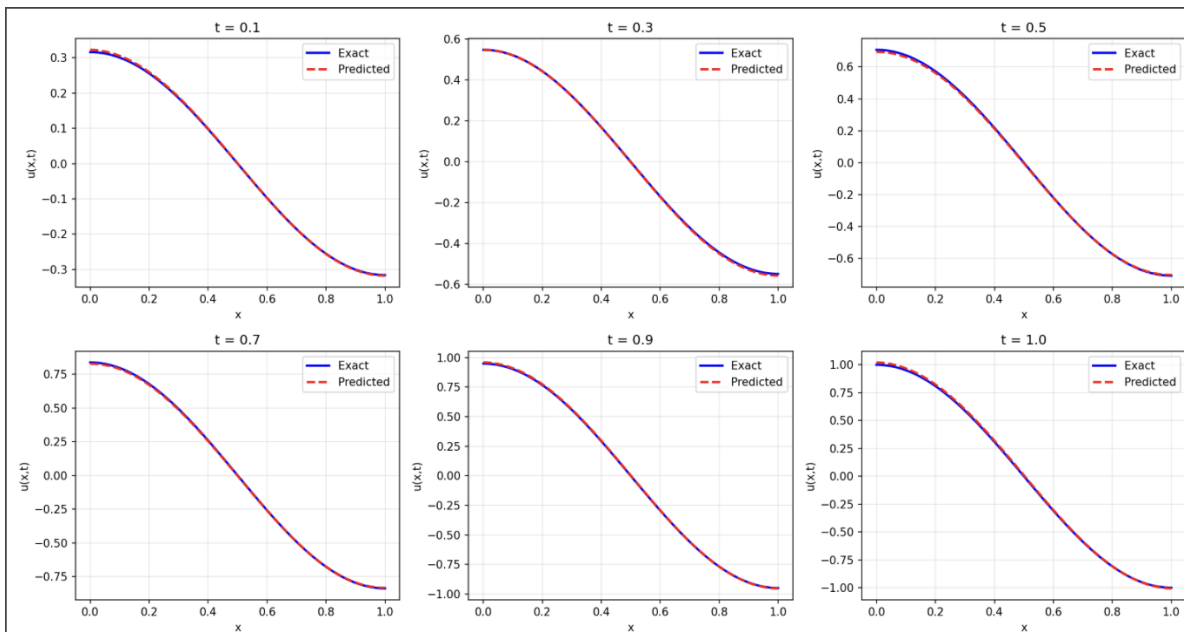


Figure 10: Product-integration fixed-time slices. These plots compare the exact and predicted solutions as functions of x for selected times and show strong agreement throughout the domain.

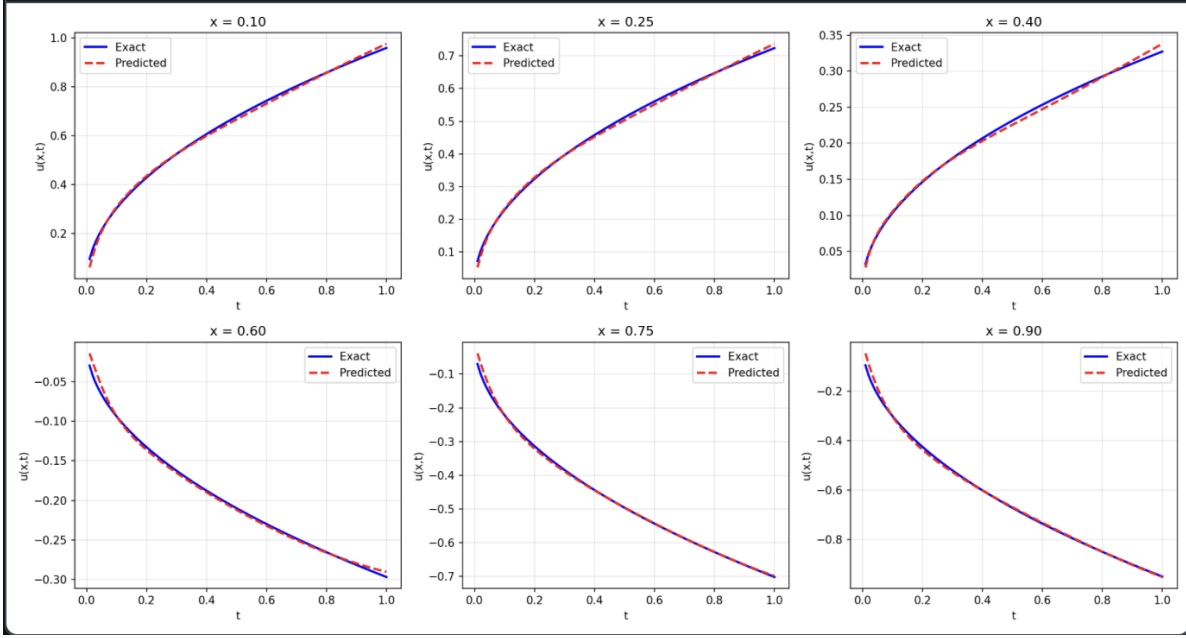


Figure 11: Product-integration fixed-position slices. These curves compare the exact and predicted solutions as functions of t for selected spatial locations and show that the network captures both the positive and negative solution regions well.

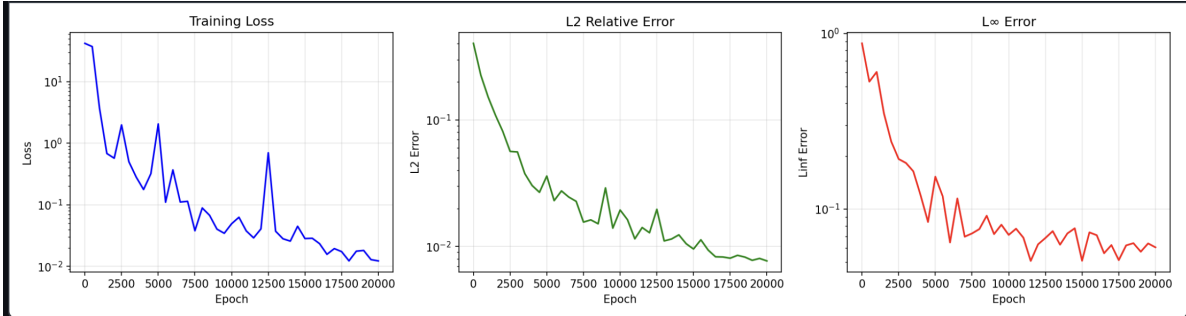


Figure 12: Training progress for the integro-differential product-integration branch. The left panel tracks the total loss together with PDE, BC, and IC loss components, while the right panel tracks the L_2 relative error and L_∞ error across training.

12.4 Early stopping analysis

The README and early-stopping guide analyze 40 checkpoints from the current branch. The key entries are summarized below.

Criterion	Epoch	Relative L_2	L_∞
Best relative L_2	20,000	1.52%	0.0606
Best L_∞	11,500	2.80%	0.0507
Best balance	17,500	1.60%	0.0512

This suggests that epoch 17,500 may be a strong practical stopping point when balancing training time and performance. In other words, the framework is now mature enough that checkpoint

selection itself becomes an optimization problem.

13 Discussion

The strongest aspect of the work so far is that it is not treating PINNs as a black-box machine-learning trick. The solver is organized around the mathematics of the equation. The graded mesh addresses the early-time singularity. The L1 scheme captures fractional memory. Product integration addresses the weak singularity in the Volterra term. The result is a theory-driven PINN rather than a purely empirical one.

Another important takeaway is that the project has already generated a *development narrative*. The Mexican-hat branch demonstrates representation quality and training behavior. The baseline integro-differential branch demonstrates that the full fractional PINN workflow can succeed. The product-integration branch then improves the mathematical treatment of the singular kernel and scales the problem to a much larger grid. This progression is valuable for a mid-semester report because it shows both breadth and depth.

14 Future Directions

From the project handouts and current research direction, several next-step themes naturally emerge.

14.1 Spiking neural networks for physics-informed learning

One future direction is to study whether **spiking neural networks** can be incorporated into a physics-informed framework. Since spiking models are event-driven and can be more energy-efficient than standard dense neural networks, they may offer advantages for solving structured scientific problems where sparsity and temporal dynamics are important. For fractional equations in particular, this raises interesting questions about whether memory effects can be represented efficiently in spike-based architectures.

14.2 Quantum PINNs

A second direction is **quantum PINNs**. The core idea would be to investigate whether hybrid quantum-classical architectures can be used to approximate PDE or integro-differential solution operators. While this is still exploratory, it aligns well with the broader theme of next-generation computational methods for scientific machine learning. For this project, a future study could examine whether parameterized quantum circuits can be combined with physics-informed losses in a stable way.

14.3 Applications to LEO and MIMO systems

Another long-term direction is to connect PINN methodology to engineering applications such as **low Earth orbit (LEO) communication systems** and **massive MIMO systems**. These application areas often involve propagation, interference, dynamic environments, and inverse problems. The current project on fractional and memory-driven equations provides useful training because it develops tools for handling nonlocality, complex dynamics, and constrained learning. In later stages, that methodology could be adapted to communication-system modeling, parameter estimation, channel prediction, or dynamic system identification.

15 Conclusion

This mid-semester report documents the development of a PINN framework for fractional differential and fractional integro-differential equations. The project has progressed through multiple meaningful stages: a Mexican-hat branch that demonstrates the expressive power of the chosen architecture, a baseline integro-differential solver without product integration, and a current enhanced branch that uses kernel-aware product integration on a much larger grid.

The central contribution of the semester so far is the combination of neural-network approximation with fractional numerical analysis. Rather than applying a PINN in a generic way, the project integrates graded meshing, L1 history discretization, and product integration into the residual construction itself. This gives the work a strong mathematical foundation and makes it a compelling starting point for the broader theme of *PINNs and Their Applications*.